



FRONTEND SYSTEM DESIGN DOCUMENT

React SPA Architecture — ShuwaDilu E-Commerce Platform

Document Version	1.0.0 Frontend Edition
Date	August 1, 2026
Prepared By	Arsene MUSAFIRI INGENZI
Scope	Frontend architecture only
Challenge	ICT Chamber Challenge 1, E-Commerce Platform

Table of Contents

Table of Contents.....	2
1. Executive Summary.....	4
2. Platform Goals & Requirements.....	4
2.1 Functional Requirements.....	4
2.2 Non-Functional Requirements.....	4
3. High-Level Frontend Architecture.....	5
3.1 Architecture Layers.....	5
3.2 Data Flow.....	5
4. Frontend Technology Stack.....	5
5. Application Entry & Provider Architecture.....	6
5.1 Entry Point Hierarchy.....	6
5.2 Routing Map.....	6
5.3 Component Tree.....	7
6. State Management.....	7
6.1 Currency Context.....	7
6.2 Cart Context.....	7
6.3 Wishlist Context.....	8
6.4 Context Consumption Map.....	8
7. API Layer (Frontend Consumption).....	8
7.1 Current Data Source: DummyJSON.....	8
7.2 Product Data Model (TypeScript).....	9
8. Design System.....	9
8.1 Color Tokens.....	9
8.2 Typography.....	10
8.3 Spacing Scale (8pt Grid).....	10
8.4 Design Token Export.....	10
9. UI Component & Interaction Patterns.....	10
9.1 Buttons.....	10
9.2 Cards.....	10
9.3 Badges & Pill Tags.....	10
9.4 Navigation.....	11
9.5 Forms & Inputs.....	11
9.6 Iconography.....	11
10. Key UX Features & Implementation.....	11
10.1 Floating Cart Action Button.....	11
10.2 Animated Page Loader.....	11
10.3 Scroll-to-Top on Navigation.....	12
10.4 Mobile-Responsive Navbar.....	12
10.5 Currency Switcher.....	12

11. Frontend File Structure	13
12. Performance Strategies	14
13. Appendix — Design Token Reference	14

1. Executive Summary

ShuwaDilu is a B2B + B2C hybrid marketplace platform inspired by Alibaba.com, tailored for the Rwandan and East African market. The platform enables consumers, retailers, and manufacturers to discover, browse, and purchase products across multiple categories through a localized Rwandan experience supporting RWF, USD, and EUR currencies with a fully mobile-first, responsive design.

This document describes the current frontend architecture of the ShuwaDilu platform: a React single-page application built with Vite and TypeScript, styled according to the official ShuwaDilu Brand Guidelines (Sea Green, Evergreen, and Malachite palette on a Light Mint canvas, set in Montserrat). It is scoped exclusively to the client-side application routing, component structure, state management, API consumption, design system, and performance strategy.

2. Platform Goals & Requirements

2.1 Functional Requirements

#	Requirement
F1	Browse products across multiple categories
F2	Search products by name, brand, or category
F3	View detailed product information with images, specs, and attributes
F4	Add and remove products from a persistent Shopping Cart
F5	Save and unsave products to a Wishlist (Favorites)
F6	Switch display currency between RWF, USD, and EUR
F7	View Categories overview and filter products by category
F8	Full mobile-responsive experience with slide-out drawer navigation
F9	Animated branded page loader on application initialization
F10	Floating cart shortcut button visible while shopping

2.2 Non-Functional Requirements

#	Requirement
N1	Sub-2s initial page load on desktop and mobile
N2	Offline-resilient state via localStorage persistence
N3	Zero horizontal scrollbar overflow on any viewport
N4	WCAG 2.1 AA color contrast compliance (4.5:1 minimum)
N5	Scroll-to-top behaviour on every route navigation
N6	TypeScript strict-mode compilation with zero errors
N7	Exportable, Figma-ready design token system

3. High-Level Frontend Architecture

The frontend is a self-contained React SPA that communicates with an external product catalog API and persists all user state client-side. No backend service is required for the current release; the architecture below is intentionally decoupled so a proposed backend could later replace the **DummyJSON** data source without changing the presentation layer.

3.1 Architecture Layers

Layer	Responsibility
User (Browser)	Renders the SPA; all interaction happens client-side
App Router (App.tsx)	Mounts providers, routes, and global chrome (Navbar / Footer)
Context Providers	Currency → Cart → Wishlist: supply global state to every page
Pages Layer	Home · Product Details · Categories · Cart · Wishlist
Component Layer	Navbar · ProductCard · FloatingCart · PageLoader · Footer
Design System	CSS custom properties · Montserrat · ShuwaDilu palette
External API	DummyJSON REST API: https://dummyjson.com/products
Client Storage	localStorage: cart, wishlist, and currency selection

3.2 Data Flow

```
User (Browser)
├─> App.tsx (Router Root)
│   └─> Context Providers (Currency ▶ Cart ▶ Wishlist)
│       └─> Pages (Home ▶ ProductDetails ▶ Categories ▶ Cart ▶ Wishlist)
│           └─> Components (Navbar ▶ ProductCard ▶ FloatingCart ...)
│               └─> Design System (CSS Custom Properties)
```

Pages —HTTP Fetch—> DummyJSON REST API
Providers —Read/Write—> localStorage

4. Frontend Technology Stack

Layer	Technology	Reason
Framework	React 18 + Vite	Fast HMR, modern JSX, SPA routing
Language	TypeScript (strict)	Type safety, better DX, 0 runtime type errors
Styling	Vanilla CSS + Custom Properties	Maximum control, no utility bloat
Routing	React Router v6	Declarative client-side navigation
Icons	Lucide React	Consistent 1.5px-stroke vector icon set
Font	Montserrat (Google Fonts)	Premium, readable, weights 300–800

Layer	Technology	Reason
State	React Context API	Lightweight — no extra deps for cart/wishlist/currency
Persistence	localStorage	Cart, wishlist, currency survive page refresh
Build Tool	Vite	~10× faster builds than CRA
Type Checking	tsc --noEmit / tsc -b	CI-compatible strict compilation

5. Application Entry & Provider Architecture

5.1 Entry Point Hierarchy

```
main.tsx
├── BrowserRouter
│   ├── CurrencyProvider      ◀ outermost: currency affects all prices
│   │   ├── CartProvider      ◀ cart totals need currency context
│   │   │   ├── WishlistProvider
│   │   │   │   └── App.tsx
│   │   │       ├── ScrollToTop      (route change listener)
│   │   │       ├── Navbar           (sticky header)
│   │   │       ├── <Routes> ... </Routes>
│   │   │       ├── FloatingCart     (fixed bottom-right)
│   │   │       └── Footer
```

Provider order matters: `CurrencyProvider` wraps `CartProvider` so that `formatPrice()` is available globally before cart totals are computed.

5.2 Routing Map

Route	Component	Description
/	HomePage	Product grid, category pills, sort & filter
/products/:id	ProductDetailsPage	Full product specs, gallery, add-to-cart
/categories	CategoriesPage	Grid of all product categories
/categories/:slug	CategoryProductsPage	Products filtered by a specific category
/cart	CartPage	Shopping cart with quantity controls & checkout
/wishlist	WishlistPage	Saved favorite products grid

5.3 Component Tree

```

App.tsx
├── PageLoader.tsx      ◀ full-screen animated preloader (1.1s on mount)
│   └── FloatingIcons ×8 ◀ animated product icons
├── ScrollToTop.tsx     ◀ window.scrollTo(0,0) on pathname change
├── Navbar.tsx
│   ├── CategoriesMegaMenu.tsx ◀ hover dropdown mega menu
│   └── [currency dropdown]    ◀ functional RWF / USD / EUR switcher
├── <Routes>
│   ├── HomePage.tsx
│   │   ├── HeroSearch.tsx    ◀ tabbed search bar + hero section
│   │   └── ProductCard.tsx ×N
│   ├── ProductDetailsPage.tsx
│   │   ├── SkeletonLoader.tsx ◀ loading shimmer while fetching
│   │   └── [gallery, specs, supplier card, protection card]
│   ├── CategoriesPage.tsx
│   ├── CategoryProductsPage.tsx → ProductCard.tsx ×N
│   ├── CartPage.tsx
│   └── WishlistPage.tsx → ProductCard.tsx ×N
├── FloatingCart.tsx    ◀ slides up when cart has items
└── Footer.tsx
  
```

6. State Management

All shared application state lives in three React Context providers. No external state library (Redux, Zustand) is used the state surface area is small enough for Context + hooks to remain performant and easy to reason about.

6.1 Currency Context

File: `src/context/currencyContext.ts` + `CurrencyProvider.tsx` + `useCurrency.ts`

Code	Symbol	Rate (vs USD)	Label
RWF	RF	1380	English-RWF (default)
USD	\$	1.0	English-USD
EUR	€	0.92	English-EUR

A single `formatPrice(usdAmount)` function is used everywhere prices render. No dual-currency brackets are shown anywhere in the UI, in line with brand voice guidance. Selected currency is saved to `localStorage['shuwadilu_currency']`.

6.2 Cart Context

File: `src/context/cartContext.ts` + `CartProvider.tsx` + `useCart.ts`

Method	Description
<code>addToCart(product)</code>	Adds item or increments quantity
<code>removeFromCart(productId)</code>	Removes item completely
<code>updateQuantity(productId, qty)</code>	Updates quantity, removes if $qty \leq 0$

Method	Description
clearCart()	Empties entire cart
totalItems	Sum of all item quantities
totalPrice	Sum of price × quantity (USD, formatted via formatPrice)

Persistence: entire cart is saved to localStorage['cart'].

6.3 Wishlist Context

File: src/context/wishlistContext.ts + WishlistProvider.tsx + useWishlist.ts

Method	Description
toggleWishlist(product)	Adds if not present, removes if already saved
isInWishlist(productId)	Returns boolean for heart-icon fill state
totalFavorites	Count badge on navbar wishlist link

Persistence: saved to localStorage['shuwadilu_wishlist'].

6.4 Context Consumption Map

Provider	Exposes	Consumed By
CurrencyProvider	formatPrice, currency, setCurrency	Navbar, ProductCard, ProductDetailsPage, CartPage, FloatingCart
CartProvider	addToCart, removeFromCart, totalPrice, totalItems	ProductCard, CartPage, FloatingCart, Navbar
WishlistProvider	toggleWishlist, isInWishlist, totalFavorites	ProductCard, WishlistPage, Navbar, ProductDetailsPage

7. API Layer (Frontend Consumption)

7.1 Current Data Source: DummyJSON

Base URL: <https://dummyjson.com/products>

All API calls are centralized in src/api/dummyjson.ts:

Function	Endpoint	Description
fetchProducts(limit, skip, sortBy, order)	GET /products	Paginated product list with sorting
fetchProductById(id)	GET /products/:id	Single product full details
searchProducts(query)	GET /products/search?q=	Text search

Function	Endpoint	Description
fetchCategories()	GET /products/categories	All category slugs & names
fetchProductsByCategory(slug, sortBy, order)	GET /products/category/:slug	Filtered by category

All fetch calls use the native Fetch API — no axios dependency. Errors throw descriptive messages caught by component-level try/catch blocks with error-state rendering.

7.2 Product Data Model (TypeScript)

```
interface Product {  
  id: number;  
  title: string;  
  description: string;  
  category: string;  
  price: number; // stored in USD, formatted via formatPrice()  
  discountPercentage: number;  
  rating: number;  
  stock: number;  
  tags: string[];  
  brand?: string;  
  sku?: string;  
  weight?: number;  
  dimensions?: { width: number; height: number; depth: number };  
  warrantyInformation?: string;  
  shippingInformation?: string;  
  availabilityStatus?: string;  
  returnPolicy?: string;  
  minimumOrderQuantity?: number;  
  thumbnail: string;  
  images: string[];  
  reviews?: Review[];  
}
```

8. Design System

The design system implements the ShuwaDilu Brand Guidelines directly as CSS custom properties, ensuring visual consistency across every page and component.

8.1 Color Tokens

Token	Hex	Role
--color-light	#F4FBF6	App canvas background (Light Mint)
--color-evergreen	#0D2818	Headings, dark surfaces, footer
--color-black-forest	#04471C	Hover accents, button gradient dark end
--color-sea-green	#058C42	Primary brand CTA green
--color-malachite	#16DB65	Neon highlights, badges, glows (dark surfaces only)
--color-success	#059669	Stock / added-to-cart success state
--color-warning	#D97706	Star ratings
--color-danger	#DC2626	Wishlist hearts, remove actions

8.2 Typography

Role	Size	Weight	Usage
Hero Heading	36px	800	Page hero titles
Section Heading (h1)	30px	800	Page titles
Sub-heading (h2)	24px	700	Section titles
Card Title (h3)	16px	700	Product card names
Body	16px	300	Paragraph text
Caption / Label	14px	500–600	Labels, badges
Small / Fine	12px	600	Tags, metadata

Primary typeface: Montserrat (weights 300–800). Monospace typeface: Fira Code, used exclusively for technical documentation and code blocks. Body text is always Light (300) for an airy, premium feel; headings always use Bold (700) or ExtraBold (800). Prices are always ExtraBold in Sea Green. Italics are never used in headings. Body line height is 1.6.

8.3 Spacing Scale (8pt Grid)

4px → 8px → 12px → 16px → 24px → 32px → 40px → 48px → 64px

8.4 Design Token Export

Full design system tokens are exported to /public/shuwadilu-theme-tokens.json (and a project-root copy), importable into Figma via the Tokens Studio plugin, keeping design and code in sync.

9. UI Component & Interaction Patterns

9.1 Buttons

Variant	Specification
Primary CTA	Gradient #058C42 → #04471C · white text · Montserrat Bold 12px · pill radius (9999px) · hover lift + glow shadow
Secondary / Outline	White background · 1px mint border (#DCFCE7) · Evergreen text · hover border → Sea Green
Danger (Remove/Delete)	Transparent background · Danger red (#DC2626) text · hover underline

9.2 Cards

Product Card: white background, 1px mint border, 12px radius, subtle shadow (0 1px 3px rgba(15,23,42,0.05)). On hover, the card lifts 4px, the border shifts to Sea Green, and a glow shadow appears. Image area uses a 4:3 ratio with object-fit: contain and internal padding.

9.3 Badges & Pill Tags

Badge	Style
Discount Badge	Red (#DC2626) background, white text, top-left of image
Category Badge	Evergreen translucent background, white text, bottom-right, backdrop blur
Active Pill	Sea Green background, white text
Default Pill	White background, mint border, dark text

9.4 Navigation

- Sticky header: 76px height, white background, subtle shadow
- Logo always links to home (/)
- Active nav links highlighted in Sea Green
- Hover states use the elevated surface tint (#EDF7F1)

9.5 Forms & Inputs

Focused inputs use a 2px Sea Green border and pill radius (40px) for the hero search field, white background, Montserrat Regular 16px, and a glow shadow on focus (0 10px 35px rgba(5,140,66,0.28)).

9.6 Iconography

ShuwaDilu uses Lucide React as its standard icon library a consistent 24×24px grid with 1.5px stroke weight that matches Montserrat's geometric personality.

Rule	Specification
Default Size	18–20px in UI, 24px in feature highlights
Stroke Weight	1.5px (Lucide default)
Color	Inherits context : Sea Green when active, text-muted when inactive
Label Pairing	Icons are always paired with text labels in navigation
Accessibility	title or aria-label required on standalone icons

10. Key UX Features & Implementation

10.1 Floating Cart Action Button

- Trigger: appears when totalItems > 0 and the current path is not /cart
- Content: live item-count badge (neon Malachite #16DB65) + formatted subtotal price
- Animation: slideUpGlow entrance with a cubic-bezier spring curve
- Position: fixed, bottom 28px, right 28px, z-index 990

10.2 Animated Page Loader

- Fires once on app mount, auto-dismisses after 1,100ms
- Three concentric spinning emerald rings (clockwise + counter-clockwise)
- Central ShuwaDilu mark pulsing with a green drop-shadow glow

- Eight floating product icons fading in the background
- z-index 9999: renders above all content

10.3 Scroll-to-Top on Navigation

ScrollToTop.tsx listens to route (pathname) changes and calls `window.scrollTo(0, 0)` on every transition, preventing SPA scroll-position carry-over issues.

10.4 Mobile-Responsive Navbar

Breakpoint	Behaviour
> 1350px	Full desktop: Logo + All Categories + Quick Links + Widgets + Auth
≤ 1350px	Quick Links (Verified Products, Hot Deals) hidden
≤ 1150px	Desktop widgets hidden; hamburger button shown
Mobile Drawer	Slides in from right, 300px wide, <code>slideInRight</code> animation

10.5 Currency Switcher

- Functional dropdown in the Navbar with an animated `fadeInDown` menu
- Three options: RWF (default), USD, EUR
- All prices across every surface: cards, details, cart, floating cart, order summaries update instantly
- Selection persists in `localStorage['shuwadilu_currency']`
- No bracket equivalence is ever shown a single clean currency per session, per brand voice rules

11. Frontend File Structure

```
ch1-ecommerce/
├── public/
│   ├── shuwadilu-horizontal.png    ◀ brand horizontal logo
│   ├── shuwadilu-mark.png         ◀ brand mark (used in preloader)
│   └── shuwadilu-theme-tokens.json ◀ Figma design token export
├── src/
│   ├── api/
│   │   └── dummyjson.ts           ◀ fetch functions + TS interfaces
│   ├── components/
│   │   ├── CategoriesMegaMenu.(tsx/css)
│   │   ├── FloatingCart.(tsx/css)
│   │   ├── Footer.(tsx/css)
│   │   ├── HeroSearch.(tsx/css)
│   │   ├── Navbar.(tsx/css)
│   │   ├── PageLoader.(tsx/css)
│   │   ├── ProductCard.(tsx/css)
│   │   ├── ScrollToTop.tsx
│   │   └── SkeletonLoader.(tsx/css)
│   ├── context/
│   │   ├── cartContext.ts / CartProvider.tsx / useCart.ts
│   │   ├── currencyContext.ts / CurrencyProvider.tsx / useCurrency.ts
│   │   └── wishlistContext.ts / WishlistProvider.tsx / useWishlist.ts
│   ├── pages/
│   │   ├── CartPage.(tsx/css)
│   │   ├── CategoriesPage.(tsx/css)
│   │   ├── CategoryProductsPage.(tsx/css)
│   │   ├── HomePage.(tsx/css)
│   │   ├── ProductDetailsPage.(tsx/css)
│   │   └── WishlistPage.(tsx/css)
│   ├── App.tsx                    ◀ router + provider integration + preloader
│   ├── App.css
│   ├── main.tsx                   ◀ React DOM root + provider wrapping
│   └── index.css                  ◀ global design system tokens + resets
├── shuwadilu-theme-tokens.json    ◀ design token export (root copy)
├── package.json
├── tsconfig.json
└── vite.config.ts
```

12. Performance Strategies

Strategy	Implementation
Lazy Image Loading	loading="lazy" on all tags in product cards
Client-Side State Caching	Cart, wishlist, and currency persist in localStorage: no re-fetch on reload
Minimal Dependencies	No Redux, no axios, no UI component library: pure React + Fetch API
Skeleton Loaders	SkeletonLoader.tsx shown during product-detail fetch
CSS Containment	overflow-x: hidden on html, body, #root, .app prevents layout thrashing
Vite Build	Tree-shaking + code splitting by default via the Rollup bundler
Font Preloading	Google Fonts loaded with display=swap to prevent FOIT

13. Appendix — Design Token Reference

Full token export available at </public/shuwadilu-theme-tokens.json>.

Token	Value	Usage
--color-sea-green	#058C42	Primary CTA, prices, active tabs
--color-evergreen	#0D2818	Headings, footer background
--color-malachite	#16DB65	Badges, neon glows
--color-light	#F4FBF6	Page canvas
--radius-full	9999px	Pill buttons
--radius-lg	12px	Cards
--shadow-glow	0 0 20px rgba(5,140,66,0.18)	Brand green glow
--transition-fast	0.15s ease	Hover states
--transition-normal	0.25s ease	Panel / card animations

*ShuwaDilu, Empowering Rwandan Trade.
Built for ICT Chamber Challenge 1.*